

AIE683 Machine Learning

Week 12: The ML Lifecycle — From Model to Production

Dr. Hakan Emekci

Hacettepe University, Graduate School of Informatics

A model that worked, until it did not

- **Zillow Offers (2018–2021):** Zillow bought homes directly, priced with its valuation models, and resold them
- In 2021 the housing market shifted quickly; the company bought many homes above what it could later sell them for
- Q3 2021: an inventory write-down of **USD 304 million** (with USD 240–265 million more expected in Q4); in November 2021 the business was wound down and about a quarter of staff let go
- Not only a modelling failure (operations, incentives, market speed), but a clear lesson:

a model's accuracy is a property of the model *and* the world it is applied to

- Closer to home: many demand, credit and fraud models broke in spring 2020, when behaviour changed overnight

Plan for today

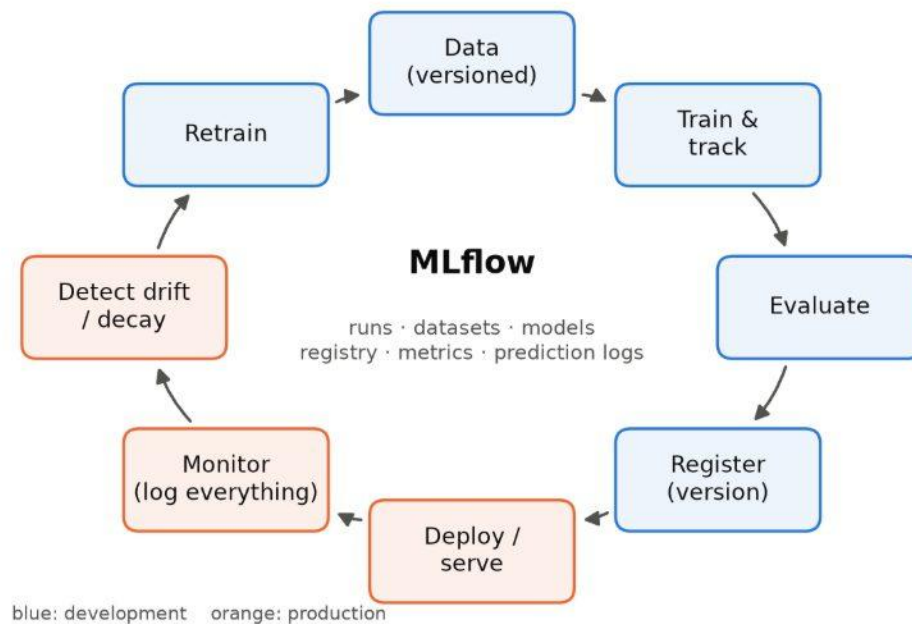
1. The lifecycle as a loop (15 min)
2. Packaging: MLflow Models, signatures, trusted types (25 min)
3. Model Registry: versions and aliases (20 min)
4. Serving: batch vs. online, `mlflow models serve`, FastAPI (25 min)
5. *Break*
6. Monitoring and drift: what to log, KS, PSI, two-sample tests (35 min, **Try it 1**)
7. Anomaly detection as a drift signal (15 min)
8. Retraining and promotion, CI/CD for ML (25 min, **Try it 2**)
9. Real data: a drift detective case (15 min, **Try it 3**)

A model is not static

data → train → evaluate → register → deploy → monitor → retrain → ...

- The world changes: **data drift** (inputs) and **concept drift** (the input→output relation)
- A model that scored well last month can degrade **silently**: no exception, no error log, just worse decisions
- Production ML = a loop, not a notebook
- Everything in the loop is **logged and versioned**: data, code, parameters, models, predictions

The lifecycle as a loop



What goes wrong in production

- **Training–serving skew:** preprocessing done in the notebook but re-implemented (differently) in the service
- **Schema changes upstream:** a column renamed, a unit changed (TL → thousands of TL), a category added
- **Data drift:** the population changes (new market, price change, season)
- **Concept drift:** the same inputs now mean something else (competitor, policy, pandemic)
- **Feedback loops:** the model's own decisions change future training data (who got a loan, who got a retention offer)
- **Delayed or missing labels:** you learn you were wrong weeks later, or never

Packaging models: MLflow Models

What is an MLflow Model?

A directory with a fixed layout:

```
model/
├── MLmodel          # flavors, signature, how to load
├── model.skops      # the fitted estimator (scikit-learn flavor)
├── requirements.txt  # pip environment
├── python_env.yaml / conda.yaml
├── input_example.json
└── serving_input_example.json
```

- **Flavors:** the same model can be loaded as `mlflow.sklearn` (native object) or `mlflow.pyfunc` (generic `predict(DataFrame)`)
- `pyfunc` is what serving tools use: one interface for scikit-learn, XGBoost, PyTorch, custom code
- In MLflow 3 a logged model is a first-class object with its own id: `models:/m-<id>`

Signatures and input examples

```
from mlflow.models import infer_signature

with mlflow.start_run() as run:
    model.fit(X_train, y_train)
    signature = infer_signature(X_train, model.predict_proba(X_train).
    info = mlflow.sklearn.log_model(
        model, name="model", signature=signature,
        input_example=X_train.iloc[:5],
        pyfunc_predict_fn="predict_proba",          # serve probabilities
        skops_trusted_types=[
            "sklearn.ensemble._hist_gradient_boosting.predictor.TreeP
    )
```

- A logged model = artifact + environment + **signature** (input/output schema)
- Load anywhere: `mlflow.pyfunc.load_model(info.model_uri)`

Schema enforcement

```
model = mlflow.pyfunc.load_model("models:/churn-classifier@champion")
model.metadata.get_input_schema()
# ['tenure_months': double, 'monthly_charges': double, 'support_calls'

model.predict(X.drop(columns="support_calls"))
# MlflowException: Failed to enforce schema ... Model is missing input
```

- Missing columns and incompatible types fail **loudly**, before the model sees the data
- Integer columns cannot hold missing values: if a feature may be missing in production, declare it as float at training time
- Schema checks catch *broken* data, not *shifted* data: drift needs statistics

Model files are code: trusted types

- Pickle files can execute arbitrary code when loaded
- MLflow 3 saves scikit-learn models with **skops**, which refuses unknown types by default
- Some estimators (e.g. `HistGradientBoosting*`) need their internal types listed explicitly: `skops_trusted_types=[...]`
- Only trust types for models **you** produced; never load model files from unknown sources
- Pin the environment: `pip_requirements=[...]` or let MLflow infer it (in a uv project it exports the lock file)

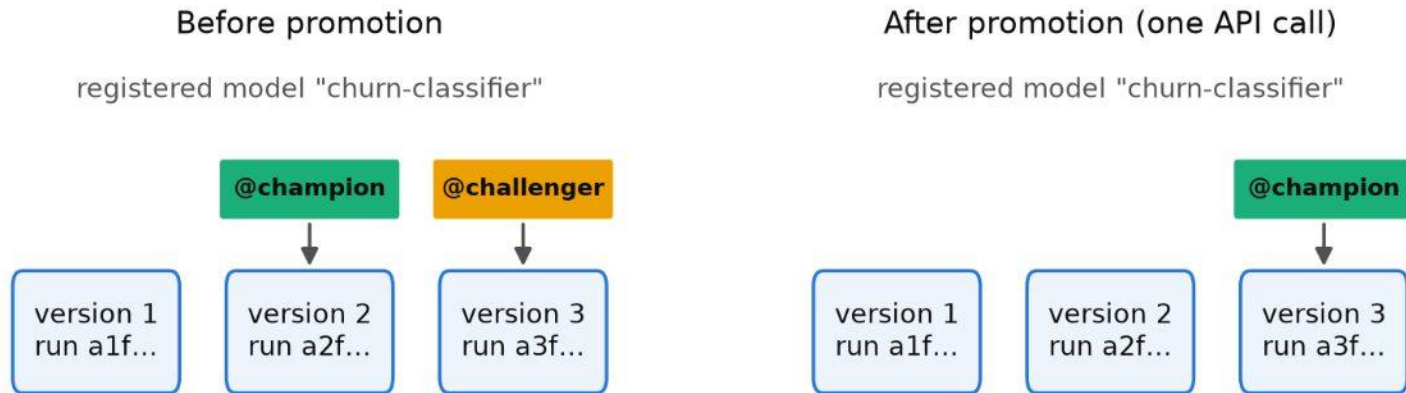
Model Registry: versions and aliases

Registering a model

```
mv = mlflow.register_model(info.model_uri, "churn-classifier")    # → version :  
  
client = mlflow.MlflowClient()  
client.set_registered_model_alias("churn-classifier", "champion", mv.version)  
client.set_model_version_tag("churn-classifier", mv.version, "trained_on", "mor  
  
model = mlflow.pyfunc.load_model("models:/churn-classifier@champion")  
client.get_model_version_by_alias("churn-classifier", "champion").version    # -
```

- Every registered model has numbered, immutable **versions**, each linked to the run that produced it
- **Aliases** (champion, challenger) are movable pointers; **tags** carry metadata (validation status, data window)
- The fixed stages "Staging / Production" of older MLflow versions are deprecated: use aliases

Promotion is moving a pointer



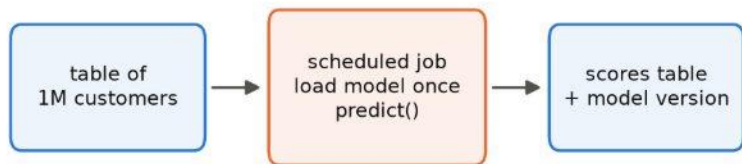
serving code always loads `models:/churn-classifier@champion`

- Rollback = move the alias back: `set_registered_model_alias(name, "champion", old_version)`
- Old versions stay available for audits: "which model made this decision in March?"

Serving

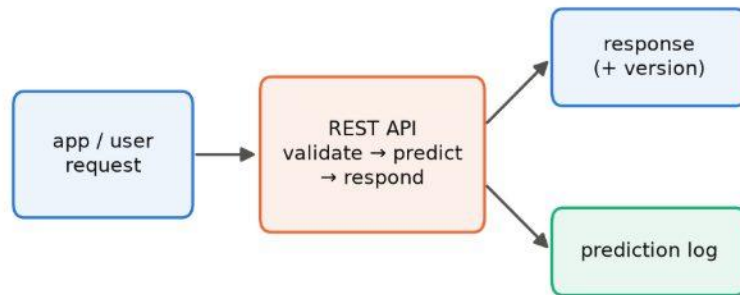
Batch vs. online inference

Batch inference (e.g. nightly)



throughput matters, latency does not

Online inference (per request)



latency (ms) and availability matter

- **Batch:** score a table on a schedule (nightly churn scores for the retention team); simple, cheap, easy to re-run
- **Online:** score each request as it arrives (fraud check at payment time); needs low latency, high availability
- Choose batch unless someone needs the answer **before** the next batch run

Built-in REST server

```
mlflow models serve -m "models:/churn-classifier@champion" -p 5001 --
```

```
curl -X POST http://127.0.0.1:5001/invocations \  
  -H "Content-Type: application/json" \  
  -d '{"dataframe_records": [{"tenure_months": 12.0, "monthly_charges"  
    "support_calls": 2, "num_products": 1, "contract_yearly": 0, "
```

- Endpoints `/invocations` (also accepts `dataframe_split`, `inputs`) and `/ping`
- `--env-manager local` reuses the current environment; the default builds the model's own environment
- `mlflow models build-docker -m ... -n churn-service` packages the same server as a container image

Your own API with FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

class Customer(BaseModel):
    tenure_months: float = Field(ge=0)
    monthly_charges: float = Field(ge=0)
    support_calls: int = Field(ge=0)
    num_products: int = Field(ge=1)
    contract_yearly: int = Field(ge=0, le=1)
    age: float = Field(ge=18)

def create_app(name: str, alias: str = "champion") -> FastAPI:
    version = client.get_model_version_by_alias(name, alias).version
    model = mlflow.pyfunc.load_model(f"models://{name}/{version}") # load once
    app = FastAPI()

    @app.post("/predict")
    def predict(customers: list[Customer]) -> dict:
        X = pd.DataFrame([c.model_dump() for c in customers])[FEATURES]
        proba = model.predict(X)[:, 1]
        log_predictions(X, proba, version) # see "monitoring"
        return {"model_version": version, "churn_probability": proba.tolist()}
    return app
```

Testing the API without a server

```
from fastapi.testclient import TestClient

api = TestClient(create_app("churn-classifier"))

r = api.post("/predict", json=[{"tenure_months": 12.0, "monthly_charg
                                "support_calls": 2, "num_products": 1
                                "contract_yearly": 0, "age": 35.0}])
assert r.status_code == 200 and "churn_probability" in r.json()

r = api.post("/predict", json=[{..., "contract_yearly": 3}])
assert r.status_code == 422 # rejected by pydantic, model ne
```

- The same tests run in CI on every commit
- Test the contract: valid input → 200, invalid → 422, response contains the version

Serving checklist

- Load the model **once**; resolve the alias at startup; report the version (/health)
- Validate inputs twice: domain rules (pydantic) and schema (MLflow signature)
- Ship preprocessing **inside** the model (Pipeline), never re-implement it in the service
- Measure latency and throughput; batch requests when possible
- Log every prediction (next section)
- Plan how a new version is picked up: restart, reload endpoint, or a new deployment

Monitoring and drift

What to log in production

What	Why
Request id, timestamp	join with labels later; debugging
Model name + version	which model made this decision
Input features (as received)	drift detection, reproducing a prediction
Output: score, class, threshold used	prediction drift, audits
Latency, errors, rejected requests	service health
Label when it arrives (delayed)	real performance over time
Data/feature pipeline version	find upstream changes

- Respect privacy: log only what you are allowed to keep; hash or drop identifiers
- In the lab the log is a list; in production a table, a message queue or an MLflow trace store

Logging predictions

```
PREDICTION_LOG = []

def log_predictions(X, proba, version):
    request_id = str(uuid.uuid4())
    now = datetime.now(timezone.utc).isoformat()
    for row, p in zip(X.to_dict("records"), proba):
        PREDICTION_LOG.append(**row, "churn_score": float(p), "model"
                               "request_id": request_id, "timestamp":

# a monitoring job, e.g. once per month:
with mlflow.start_run(run_name="monitoring"):
    mlflow.log_metrics(drift_metrics, step=month)
    mlflow.log_table(pd.DataFrame(PREDICTION_LOG), artifact_file="pre
```

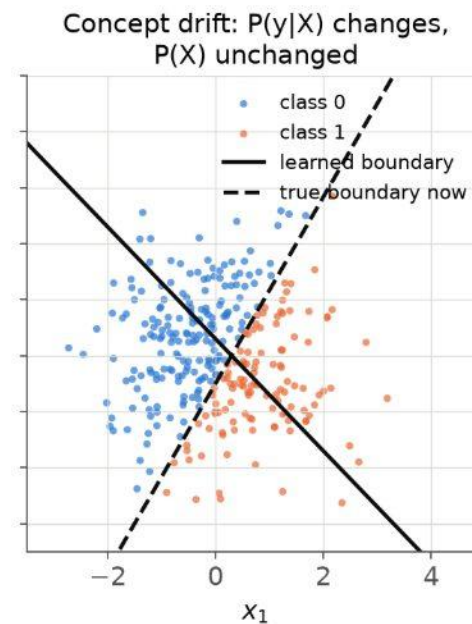
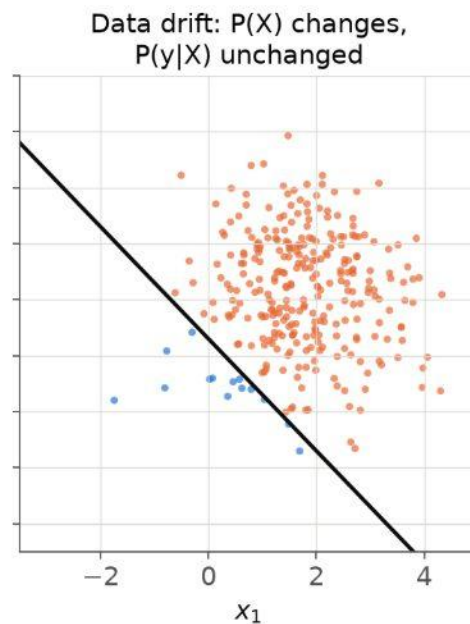
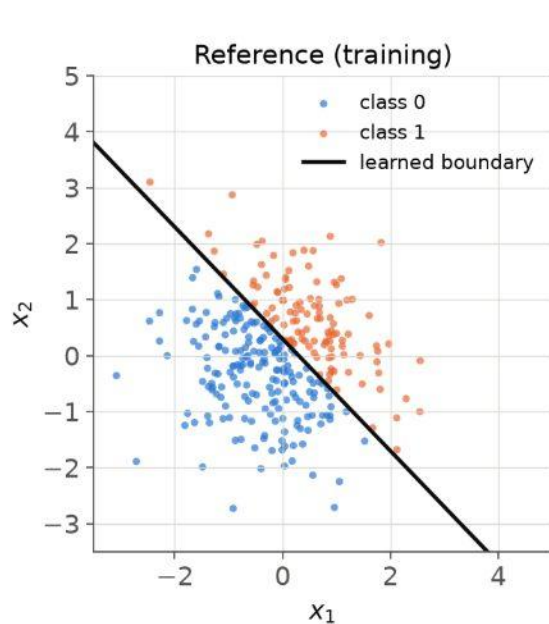
Kinds of change

Write the joint distribution as $P(X, y) = P(y | X) P(X)$.

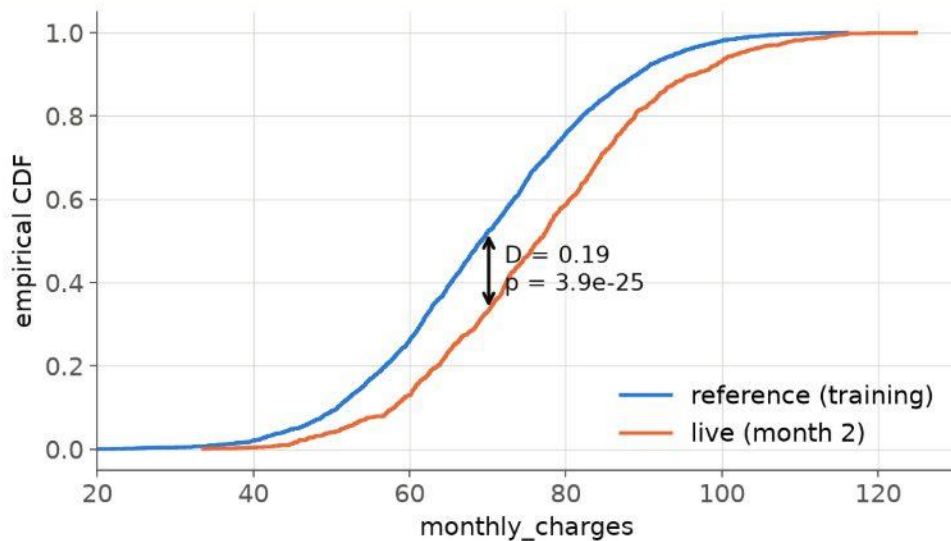
- **Data drift / covariate shift:** $P(X)$ changes, $P(y | X)$ stays *prices rise, more support calls*
- **Label (prior) shift:** $P(y)$ changes, $P(X | y)$ stays *a disease becomes more common*
- **Concept drift:** $P(y | X)$ changes *a competitor now targets loyal customers: the same customer is riskier*

Over time, change can be **sudden** (policy change), **gradual** (inflation), or **recurring** (seasons).

Data drift vs. concept drift



The two-sample KS test



$$D = \sup_x |F_{\text{ref}}(x) - F_{\text{live}}(x)|$$

```
from scipy.stats import ks_2samp
res = ks_2samp(X_train["monthly_charges"], X_live["monthly_charges"])
res.statistic, res.pvalue
```

Statistical vs. practical significance

- With thousands of rows per day, the KS test rejects for **tiny** shifts: p-values of 10^{-25} are routine
- With 20 features tested daily, some tests reject by chance ($\alpha = 0.05 \rightarrow$ about one false alarm a day)
- Monitor the **effect size** (D, PSI) and alert on a threshold that matters for the business
- Calibrate thresholds on **reference data**: how large is the statistic when nothing changed?

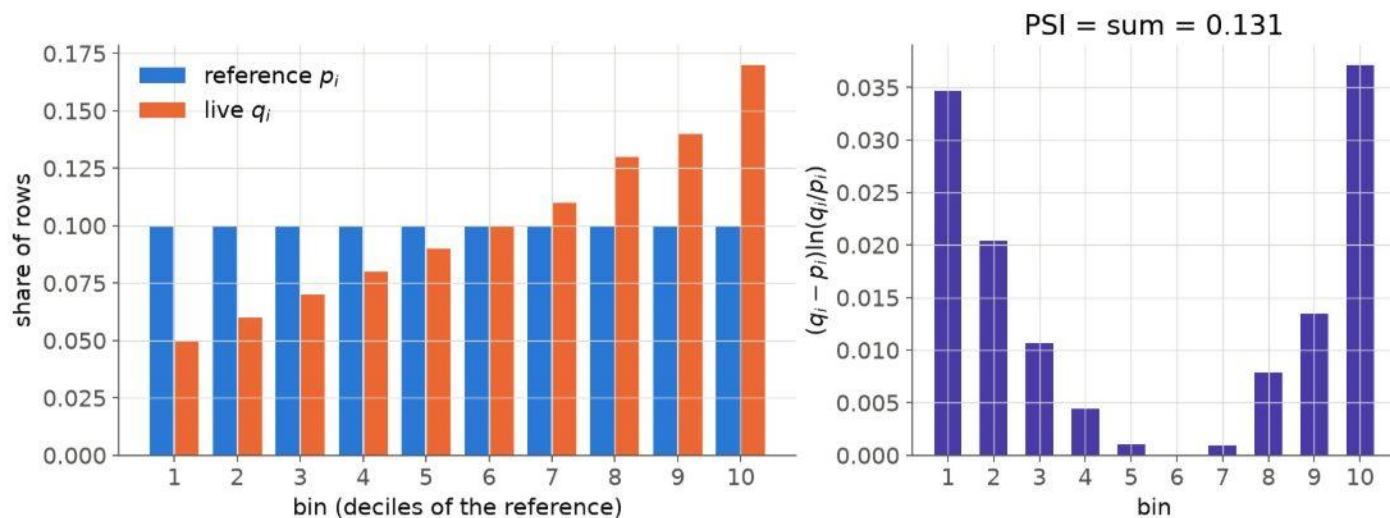
Population Stability Index (PSI)

Bin the reference data (e.g. deciles), compute the share of rows per bin: p_i (reference), q_i (live).

$$\text{PSI} = \sum_{i=1}^B (q_i - p_i) \ln \frac{q_i}{p_i}$$

- A symmetrised KL divergence: $\text{KL}(q \parallel p) + \text{KL}(p \parallel q)$ on the binned distributions
- Usual rule of thumb: < 0.1 stable, $0.1\text{--}0.25$ moderate shift, > 0.25 large shift
- Clip empty bins ($q_i = 0$) to a small ε ; use one bin per value for discrete features
- Depends on the number of bins and the sample size: the rule of thumb is a convention, not a law

Worked example: PSI by hand



bin	1	2	3	4	5	6	7	8	9	10
p_i	.10	.10	.10	.10	.10	.10	.10	.10	.10	.10
q_i	.05	.06	.07	.08	.09	.10	.11	.13	.14	.17

Bin 1: $(0.05 - 0.10) \ln(0.05/0.10) = 0.035$... bin 10: $0.07 \ln 1.7 = 0.037$.

PSI = 0.131: moderate shift.

Classifier two-sample test

Can a classifier tell reference rows from live rows?

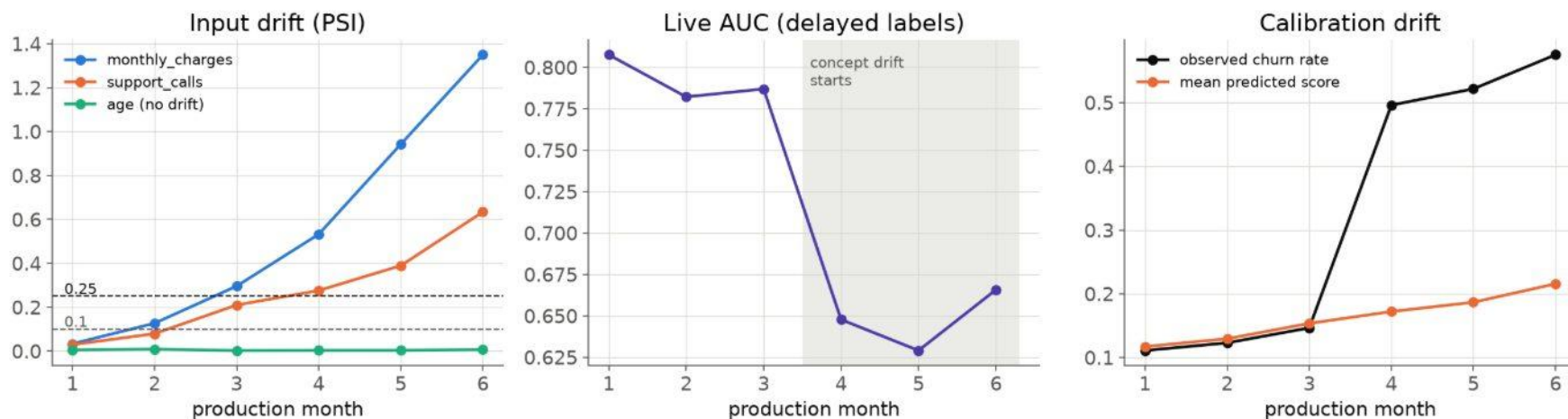
```
X = pd.concat([X_ref_sample, X_live_sample])
y = np.r_[np.zeros(len(X_ref_sample)), np.ones(len(X_live_sample))]
auc = cross_val_score(HistGradientBoostingClassifier(), X, y,
                      cv=5, scoring="roc_auc").mean()
```

- $AUC \approx 0.5$: indistinguishable. $AUC \rightarrow 1$: strong drift
- **Multivariate**: catches changes in correlations that per-feature tests miss
- Feature importances of the drift classifier (Week 9) show *which* features moved
- Same idea as "adversarial validation" for checking train/test splits

Monitoring the model's output

- **Prediction drift:** PSI of the score distribution: cheap, no labels needed, one number per model
- **Calibration drift:** mean predicted probability vs. observed rate once labels arrive
- **Performance drift:** AUC, Brier score, business metric on labeled data, by time window
- Labels are often **delayed** (churn: 30–90 days; credit default: months): performance is a lagging signal, input drift a leading one
- Proxy labels can help (complaints, early payments), but they are proxies

Worked drift example: the simulation



- Built in: prices +3 per month and more support calls (data drift); from month 4 a competitor targets loyal customers (concept drift)
- Input signals rise **smoothly**; the concept drift shows only in AUC and calibration

Worked drift example: the numbers

month	PSI charges	KS p charges	C2ST AUC	IsolationForest outlier rate	live AUC	churn rate / mean score
1	0.05	< 1e-4	0.54	1.4%	0.76	0.12 / 0.12
2	0.14	< 1e-4	0.58	3.0%	0.78	0.13 / 0.15
3	0.32	< 1e-4	0.63	4.2%	0.78	0.15 / 0.16
4	0.60	< 1e-4	0.71	4.8%	0.64	0.52 / 0.18
5	0.81	< 1e-4	0.76	6.7%	0.67	0.54 / 0.21
6	1.26	< 1e-4	0.82	9.4%	0.65	0.58 / 0.24

Reference: validation AUC 0.79, outlier rate 2% by construction.

Try it 1: calibrate your PSI alarm (10–15 min)

The 0.1 / 0.25 thresholds are folklore. How large is PSI when **nothing** changed?

1. **Predict first:** for 2000 customers drawn from the *reference* distribution, what PSI on `monthly_charges` do you expect? Write a number down.
2. In `lab.ipynb`, section *Try it 1*: draw 200 "no-drift months", compute PSI against `X_train`
3. Take the 99th percentile as your threshold. When does `monthly_charges` first cross it? And the 0.1 rule?
4. Repeat with `n_bins=20` and with months of 500 customers

Anomaly detection as a drift signal

Novelty detection on live inputs

```
from sklearn.ensemble import IsolationForest

detector = IsolationForest(contamination=0.02, random_state=0).fit(X_train)
baseline = (detector.predict(X_val) == -1).mean() # about 2%
outlier_rate = (detector.predict(X_live) == -1).mean()
mlflow.log_metric("live_outlier_rate", outlier_rate, step=month)
```

- Fit on reference data; flag live inputs unlike anything seen before (predict returns -1)
- Rising outlier rate → the model is **extrapolating** → investigate / retrain
- Per-row flags are also useful online: route unusual requests to a fallback or a human
- **IsolationForest**: random splits isolate unusual points in few steps; short average path length = anomaly

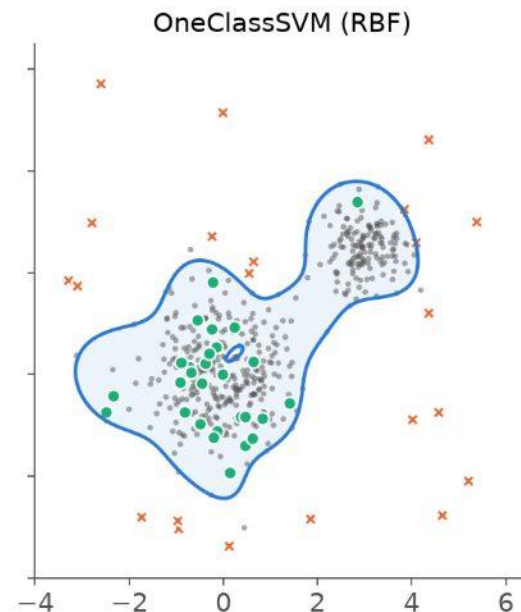
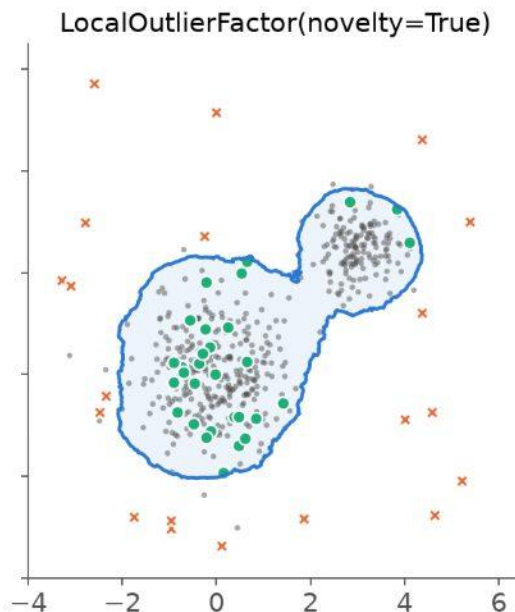
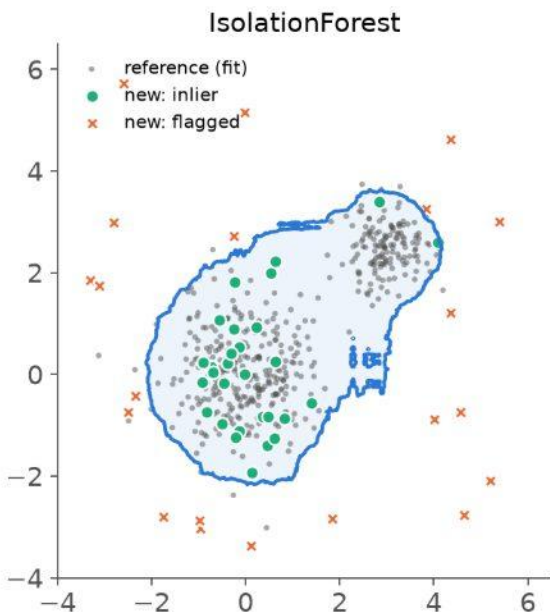
Other detectors

```
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from sklearn.covariance import EllipticEnvelope

lof = make_pipeline(StandardScaler(),
                    LocalOutlierFactor(n_neighbors=35, contamination=
ocsvm = make_pipeline(StandardScaler(), OneClassSVM(nu=0.02, gamma="s
robust = EllipticEnvelope(contamination=0.02)           # Gaussian, robust
```

- **LocalOutlierFactor:** compares local density with the neighbours' density; `novelty=True` is required to call `predict` on **new** data
- **OneClassSVM:** RBF boundary around the data, `nu` $\approx$ fraction of training points outside
- **EllipticEnvelope:** fast and interpretable if the data is roughly Gaussian
- Distance- and kernel-based methods need scaling (Week 3)

Comparing the detectors



Retraining and promotion

When and how to retrain

- **Triggers:** schedule (monthly), drift alarm, performance drop, new data source
- **Training window:**
 - *sliding*: last `k` months only: adapts fast, less data
 - *expanding*: everything so far: more data, remembers the old regime
 - *weighted*: everything, recent data up-weighted (`sample_weight`)
- Retrain = the **same pipeline code** on new data, logged as a new run, registered as a new version
- Keep a fresh, **untouched** labeled window for the comparison

Champion vs. challenger

```
champion = mlflow.pyfunc.load_model("models:/churn-classifier@champion")
challenger = mlflow.pyfunc.load_model("models:/churn-classifier@challenger")
p_a, p_b = champion.predict(X_new)[: , 1], challenger.predict(X_new)[: , 1]

diffs = []
for _ in range(1000):                                # paired bootstrap
    idx = rng.integers(0, len(y_new), len(y_new))
    diffs.append(roc_auc_score(y_new[idx], p_b[idx]) - roc_auc_score(y_new[idx], p_a[idx]))
lo, hi = np.percentile(diffs, [2.5, 97.5])

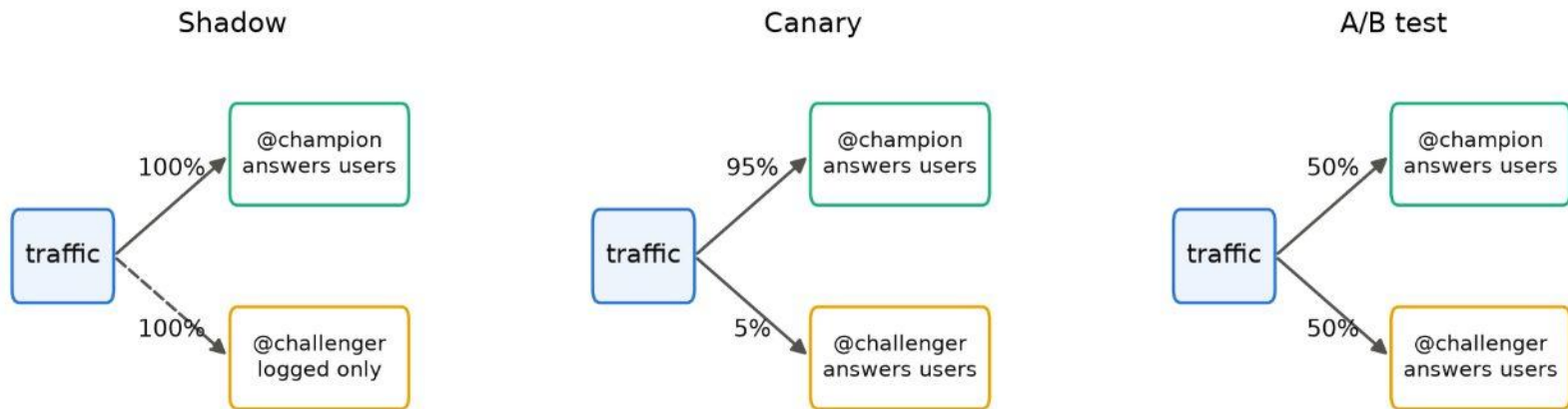
if lo > 0 and brier_score_loss(y_new, p_b) <= brier_score_loss(y_new, p_a):
    client.set_registered_model_alias("churn-classifier", "champion", challenger)
```

- Same fresh data for both, a **pre-registered** decision rule, the result logged as a run
- Lab result (month 6): AUC 0.648 → 0.698, 95% CI of the difference [0.032, 0.069], Brier 0.349 → 0.216: **promote**

Try it 2: which retraining window wins? (10–15 min)

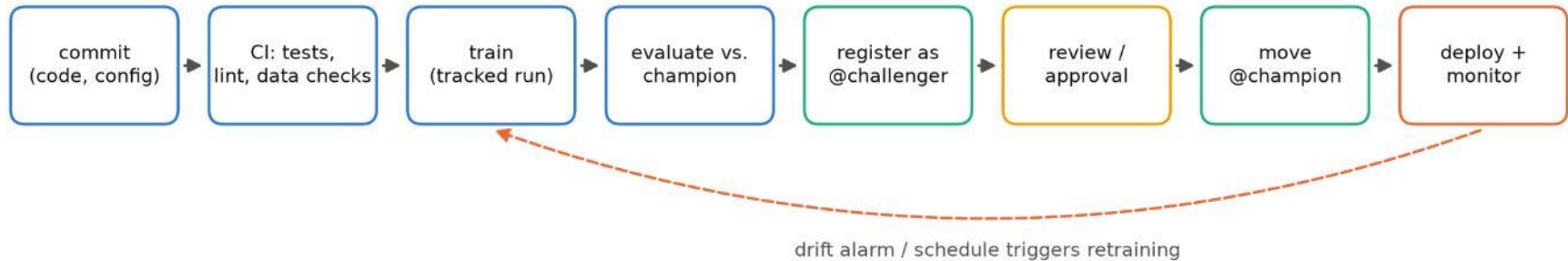
1. **Predict first:** rank *sliding* (months 4–5), *expanding* (months 0–5) and *weighted* (months 0–5, recent $\times$ 5) by AUC on month 6
2. In `lab.ipynb`, section *Try it 2*: complete the `windows` dictionary and run; each option is logged as an MLflow run
3. Compare AUC **and** Brier score. Are the AUC differences larger than the bootstrap CI from the previous slide?
4. When would the expanding window be the better choice?

Deploying a new version safely



- **Shadow:** challenger scores live traffic, answers are only logged; zero risk
- **Canary:** a small share of users gets the challenger; roll back on problems
- **A/B test:** a randomized comparison on a business metric; needs a sample-size plan
- In every case: log the version with each prediction, keep rollback one call away

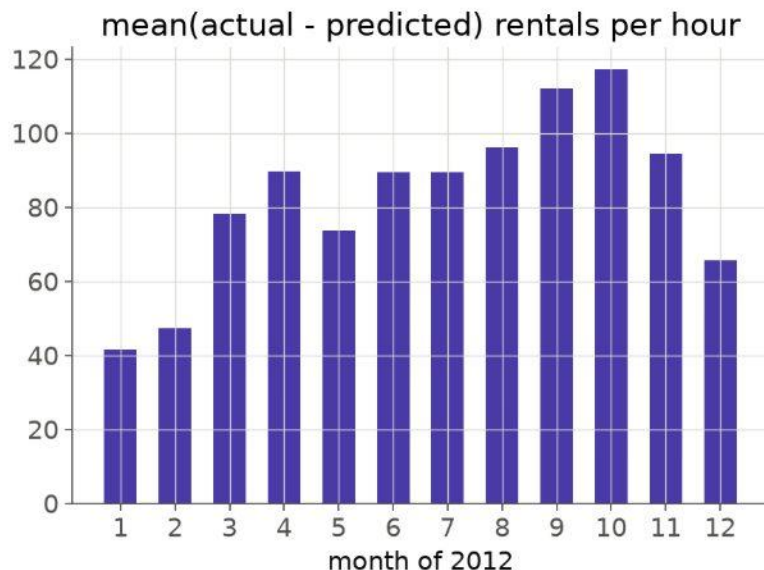
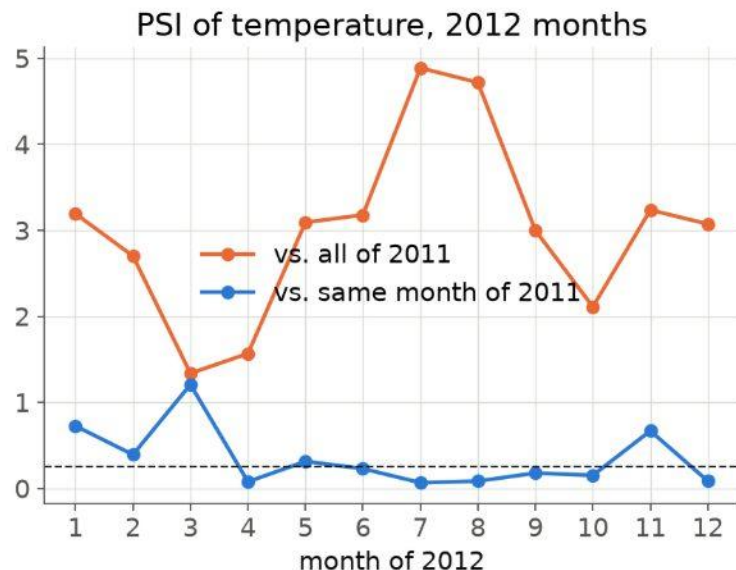
CI/CD for ML



- Version everything: code (Git), environment (`uv.lock`), data (dataset digests via `mlflow.log_input`), models (registry)
- Automated checks: unit tests, API contract tests, data validation (schema, ranges), training smoke test on a sample
- Automate up to `@challenger`; keep a **human approval** before moving `@champion` for important models

Real data: a drift detective case

Bike sharing: train on 2011, deploy in 2012



- Capital Bikeshare (Washington, D.C.), hourly rentals, the Week 11 dataset; `HistGradientBoostingRegressor` on weather and calendar features
- **Orange:** each 2012 month vs. all of 2011: $\text{PSI} > 1$ every month. **Blue:** vs. the same month of 2011
- Right: the model **under-predicts** by 40–120 rentals per hour, all year

Try it 3: drift detective (10–15 min)

In `lab.ipynb`, Part B:

1. Add `psi_temp_same_month`: PSI of each 2012 month against the **same month of 2011**. Which alarms remain?
2. Add `bias` = mean(actual – predicted) per month (labels arrive a day later: realistic)
3. Decide: data drift, concept drift, or both? *Hint*: compare mean rentals in 2011 and 2012 for the same weather
4. Log your monthly report to MLflow as a run `bike-monitoring`, one step per month
5. What would you do: retrain on a sliding window, add a trend feature, or model growth separately?

HW4: the full loop

1. Train a forecasting model (Week 11) and track it with MLflow
2. Register it, set `@champion`, and serve it (batch job or FastAPI app with tests)
3. Log predictions; feed it later or shifted data; detect the drift with calibrated signals and log them
4. Retrain, compare challenger vs. champion with a pre-registered rule and uncertainty, promote or not, and justify

Due in Week 13. Submit the notebook, the exported MLflow runs, and a short report.

Summary

- A model in production is a **loop**: package → register → serve → monitor → retrain → promote
- **MLflow Models** give a portable artifact with a signature; **skops** trusted types keep loading safe
- The **registry** separates immutable versions from movable aliases; promotion and rollback are one call
- **Log every prediction** with the model version; join labels later
- Drift: KS, **PSI**, classifier two-sample test, outlier rates; use effect sizes and **calibrated** thresholds
- Input drift is a leading signal; only labels reveal **concept drift**
- Promote with the same statistical care as any model comparison, and keep rollback ready

Backup

Where we are

- Weeks 2–11: generalization, pipelines, model families, evaluation, tuning, interpretation, unsupervised learning, forecasting
- Since Week 2 every experiment was **tracked** in MLflow (Week 8: runs, parameters, metrics, comparisons)
- What is still missing:
 - a model others can **load** without our notebook
 - a **stable name** for "the model in production"
 - a way to **answer requests**
 - evidence that the model **still works** next month
 - a **safe procedure** for replacing it

Batch scoring

```
model = mlflow.pyfunc.load_model("models:/churn-classifier@champion")
version = client.get_model_version_by_alias("churn-classifier", "cham

scores = model.predict(customers[FEATURES])[:, 1]
out = customers.assign(churn_score=scores, model_version=version,
                      scored_at=pd.Timestamp.now(tz="UTC"))
out.to_parquet("scores/2026-11-30.parquet")
```

- Resolve the alias **once** and store the concrete version with every score
- Log the batch job itself as an MLflow run (input size, model version, score summary)

Alerts that people do not ignore

- Few signals, each with a **calibrated** threshold and an **owner**
- Aggregate over a window (a week, a month), not per request
- Alert on persistent change (two consecutive windows), not on a single spike
- Separate "investigate" (input drift, outlier rate) from "act" (performance drop with labels)
- Account for **known seasonality**: compare with the same period last year, not with the whole year
- Every alert should lead to a documented decision: ignore, retrain, roll back, fix upstream

Reproducibility is the foundation

- Same code + same data + same environment + same seed $\Rightarrow$ same model
- Record per run: Git commit, `uv.lock`, dataset digest, parameters, random seeds
- Every production prediction $\rightarrow$ model version $\rightarrow$ training run $\rightarrow$ data + code
- If you cannot rebuild the current champion, you cannot safely replace it